Machine Learning-Based Predictive Analytics for Energy Consumption in Smart Grids using Big Data

Author: *Doaa Abduallah Abdeen* E-mail: *dodo_abd2005@gmail.com*
Supervised by*: Dr. Rebhi S. Baraka and is a requirement of the course Big Data (ICTS 6339), Faculty of Information Technology, Islamic University of Gaza (IUG).*

Abstract: The rapid deployment of smart grids has generated high-volume, high-velocity energy consumption data, requiring scalable big data analytics for accurate load forecasting and efficient resource management. This paper presents a comparative study of machine learning algorithms for predicting household global active power using the UCI Individual Household Electric Power Consumption dataset (2,049,280 minute-level records). Three algorithms Linear Regression, Random Forest, and Gradient Boosting—were implemented with cyclical time encoding, lag features, and rolling statistics to capture temporal dependencies. Models were trained under a chronological 80/20 split and evaluated using RMSE, MAE, R^2, and MAPE. Experiments on a single-node scikit-learn implementation showed that Gradient Boosting achieved the lowest RMSE (0.0295 kW) and R^2 (0.9989), while Linear Regression offered the fastest training (4.37 s). Random Forest achieved competitive accuracy (RMSE 0.0326 kW, MAE 0.0173 kW). A parallel implementation using Apache Spark MLlib demonstrates the distributed execution path for city-scale advanced metering infrastructure. Scalability analysis indicates increasing training cost with data volume and model complexity.

Keywords: Smart grid, load forecasting, big data, Apache Spark, machine learning, gradient boosting, random forest.

I. Introduction

The global energy sector is undergoing a radical transformation driven by renewable energy generation, the proliferation of Internet of Things (IoT) devices, and the emergence of smart grid technologies. Smart grids utilize digital communications and sensing to monitor consumption and grid conditions in near real time, thereby improving the reliability, efficiency, and sustainability of electricity distribution [1]. The deployment of Advanced Metering Infrastructure (AMI) has generated massive volumes of fine-grained consumption records. These data streams exhibit the volume, velocity, and variety characteristic of Big Data, presenting both opportunities and challenges for utilities, researchers, and end users [2].

Accurate energy consumption forecasting is essential for modern power system operation. For utilities, reliable short-term load estimates support generation scheduling, peak demand management, and stability analysis while reducing operating costs [3]. For consumers, forecasting underpins demand response, tariff design, and energy efficiency programs [4]. The growth of distributed generation and electric vehicles further increases the need for rapid, data-driven load analysis at the distribution level [5].

Traditional statistical methods, such as Autoregressive Integrated Moving Average (ARIMA) models and exponential smoothing, remain useful for stable, regular time series. However, they often underperform on high-resolution smart meter data, where consumption is shaped by nonlinear appliance usage, occupant behavior, and short-term temporal structure [6]. Machine learning regression models can learn multivariate mappings from calendar and engineered sensor features to future load without imposing a fixed functional form.

Distributed computing frameworks enable ingestion, partitioning, and modeling of AMI archives at scales that exceed single-threaded desktop workflows [7]. Apache Spark provides a fault-tolerant, in-memory analytics engine; MLlib supplies scalable learning algorithms for batch training on partitioned residential load data [8]. Mature single-node libraries such as scikit-learn remain valuable for rapid prototyping, reproducible baselines, and direct

comparison of predictive accuracy and training time on identical feature engineering workflow [9].

Among commonly used regressors, linear regression offers a low-latency interpretable baseline; random forests model nonlinear interactions and are relatively robust to outliers; and gradient boosting iteratively reduces residual error and often achieves strong accuracy on structured tabular features [10]. Algorithm selection should therefore consider not only point prediction error (e.g., root mean square error) but also training cost, scalability with sample size, and deployment constraints in operational smart grid environments [11].

Despite progress in load forecasting and Big Data analytics, gaps persist in large-scale applied studies. Many experiments use small samples or short horizons that do not reflect AMI deployment scale. Few studies report predictive metrics together with measured training runtime on both single-node and distributed engines using the same feature engineering path. Scalability is often discussed qualitatively rather than quantified with respect to data volume and model complexity.

This paper addresses these gaps through a reproducible comparative study of household global active power forecasting using a large public dataset. Specifically:

1. Implement and evaluate three regression models—Linear Regression, Random Forest, and Gradient Boosting—under a chronological train/test protocol, using a scikit-learn implementation and a parallel Apache Spark MLlib implementation;
2. Engineer temporal features, including cyclical encodings of hour, day-of-week, and month; lagged global active power and sub-meter channels; and rolling statistics to capture short-term dynamics in minute-level AMI data;
3. Conduct experiments on the UCI Individual Household Electric Power Consumption dataset (2,049,280 minute-level records after cleaning), reporting RMSE, MAE, R^2, and MAPE on a held-out test set;
4. Analyze scalability and efficiency, including data ingest throughput, feature engineering time, training duration, and Random Forest behavior under subsampled training sets and alternative hyperparameter settings;
5. Provide practical guidance on balancing prediction accuracy and computational cost in smart grid analytics.

II. Related Work

A. Smart Grid and Big Data Analytics

The integration of sensing, communication, and analytics into power distribution has accelerated data generation from smart meters, phasor measurement units, and IoT-enabled devices. These streams exhibit large volume, high velocity, and diverse variety, aligning with the Big Data paradigm and motivating platforms that go beyond single-server analytics [1], [2]. Reviews of smart-grid analytics emphasize that batch and near-real-time processing of advanced metering infrastructure (AMI) data supports load forecasting, demand response, asset management, and anomaly detection [3].

Distributed engines are commonly adopted to partition historical AMI archives and execute cleansing, aggregation, and model training at scale. Apache Spark provides resilient, in-memory computation and has become a standard substrate for city- and feeder-level energy analytics workflows [4], [5]. Prior studies report that the practical value of Big Data deployments depends not only on forecast accuracy but also on end-to-end latency, fault tolerance, and the cost of recurring model retraining as meter populations grow [2], [3].

B. Machine Learning for Energy Consumption Prediction

Electric load forecasting has been surveyed extensively across horizons (very short-term to long-term) and spatial granularities (system, feeder, customer) [6]. Classical statistical models—including ARIMA and exponential smoothing—remain competitive on smooth series but may degrade when nonlinear occupant behavior, weather variability, and distributed generation shape high-resolution consumption traces [6].

Tree-based ensemble methods are widely used for nonlinear regression on tabular energy features. Random Forests combine bagged decision trees and random subspace selection, offering robustness to outliers and implicit interaction effects among voltage, current, and sub-meter channels [7]. Gradient Boosting Machines iteratively fit weak learners to pseudo-residuals and frequently achieve strong point-forecast accuracy on structured smart-meter datasets, often at higher training cost than bagging [8]. Linear Regression continues to serve as an interpretable, low-latency baseline for comparative studies [6].

Deep learning—particularly Long Short-Term Memory (LSTM) networks—can model long-range temporal dependencies in sequential load data [9], [10]. However, several studies note that when strong autoregressive structure is present in minute-level residential series, well-engineered lag and rolling features allow tree ensembles to approach deep-model accuracy with lower implementation complexity and without specialized GPU infrastructure [6], [10]. This motivates our focus on scalable tree and linear learners with explicit temporal feature design.

Comparative studies on household smart-meter data further highlight that data preprocessing and feature

engineering strongly influence outcomes, often dominating differences among learning algorithms [6], [11]. Public archives such as the UCI Individual Household Electric Power Consumption dataset provide a reproducible benchmark for algorithm evaluation at approximately two million labeled minutes [12].

C. Distributed Machine Learning and Scalability

Zaharia et al. introduced Spark to support iterative machine learning workloads that are inefficient under MapReduce-style disk-heavy execution [4]. MLlib implements distributed versions of linear models, tree ensembles, and related learners with APIs integrated into Spark SQL and the DataFrame API [5]. Benchmarking and survey work report that distributed training benefits grow with partition count and feature dimensionality, although driver coordination, shuffles, and JVM overhead can dominate runtime on small clusters or single-node local[*] deployments [5], [13].

Empirical studies of Spark-based learning note that algorithm scalability is not uniform: tree ensembles exhibit different training-time growth with data volume and ensemble size than linear models [5], [13]. For operational smart grids, these differences inform whether nightly batch retraining, feeder-level parallelism, or edge-side baselines are appropriate. Our work quantifies training time and ingest throughput alongside RMSE and related metrics to make this trade-off explicit on a public AMI-scale dataset.

D. Feature Engineering for Time-Series Energy Data

For minute-level residential load, calendar effects (hour-of-day, day-of-week, season) and short-term persistence are dominant predictors. Cyclical encoding maps periodic variables to sine–cosine pairs so that, for example, 23:00 is close to 00:00, improving regression and tree splits [6], [11]. Lag features inject prior consumption into the input vector, capturing autoregressive structure; rolling statistics smooth noise while retaining local trend information [11]. Together, these transformations reduce the need for complex sequence architectures in single-site forecasting tasks and standardize inputs across sklearn and Spark MLlib implementations [11], [12].

E. Research Gap and Motivation

Despite extensive literature on smart grids and load forecasting, three gaps remain relevant to reproducible Big Data coursework and applied AMI analytics.

First, many comparative experiments rely on proprietary or subsampled data, limiting reproducibility. The UCI household consumption corpus is publicly available and AMI-scale, yet is not always evaluated under consistent chronological protocols with unified feature engineering workflow [12].

Second, studies often emphasize accuracy alone without reporting training time, throughput, and scalability on both single-node and distributed engines using identical features [5], [13].

Third, guidance for selecting among linear, bagging, and boosting models under joint accuracy–latency constraints is rarely grounded in measured runtime on the same dataset [6], [8].

III. METHODOLOGY

A. Dataset Description

This study uses the Individual Household Electric Power Consumption dataset from the UCI Machine Learning Repository [12]. The archive records minute-level measurements from a single dwelling in Sceaux, France, spanning approximately 47 months (December 2006–November 2010).

The dataset provides nine raw fields: date, time, global active power, global reactive power, voltage, global current intensity, and three sub-metering channels (kitchen, laundry, and water heater/air conditioning). The target variable is Global_active_power (kilowatts), representing total minute-averaged active demand.

TABLE I
Dataset Characteristics

Attribute	Value
Total records	2,049,280
Time period	Dec 2006 – Nov 2010
Sampling rate	1 minute
File size	~127 MB (uncompressed text)
Geographic location	Sceaux, France
Missing values (raw)	~1.25% (encoded as ?)
Clean records	2,049,256

B. Data Preprocessing

The raw file, separated by semicolons, underwent multi-stage processing to ensure its suitability for machine learning. Records containing missing values in the baseline fields were deleted, resulting in a clean dataset of 2,049,256 minute-level observations (from approximately 2.05 million raw readings).

The date and time fields were collected and then analyzed into a single date and time index to support the extraction of time features (hour, day of the week, month, and periodic encodings).

All numeric fields were converted to floating-point representations. Global active power ranged from 0.076 kW to 11.122 kW, with an average consumption of 1.092 kW.

The distribution exhibits a rightward skew, characteristic of residential loads: most minutes show low baseline demand with intermittent periods of power spikes.

C. Feature Engineering

An integrated feature engineering path was developed to track time-related patterns and hidden connections in electricity consumption data. The final feature set comprised 23 designed features, divided into five categories:

1) Key Measurement Features: The first six measurement features (Global_reactive_power, Voltage, Global_intensity, Sub_metering_1, Sub_metering_2, Sub_metering_3) were retained as the core features to provide essential information about the electrical status of the dwelling.

2) Time-Recurring Features: Time-related features were encoded via periodic transformations to maintain chronological continuity. For the daytime (0-23), day of the week (0-6), and month (1-12) features, sine and cosine transformations were applied.

$$x_{sin} = \sin\left(\frac{2\pi x}{x_{max}}\right), \quad x_{cos} = \cos\left(\frac{2\pi x}{x_{max}}\right) \tag{1}$$

This coding ensures that similar values have identical representations. A binary attribute (is_weekend) was also introduced to differentiate between weekday and weekend energy consumption patterns.

3) Delay Attributes: Delay attributes were generated to monitor time-related correlations in energy consumption. Past values for Global_active_power and Sub_metering_1 were inserted at 1, 3, 6, and 24 minutes, providing information about recent consumption history that may influence current consumption.

4) Moving Window Statistics: Moving window statistics were calculated to monitor consumption trends and fluctuations. Moving averages were calculated at 3, 6, and 24 minutes window sizes for Global_active_power, providing smoothed visualizations of short-term and daily consumption patterns.

5) Extracted Attributes: The Sub metering Total Attribute. This attribute was derived by summing the three sub metering channels, representing the portion of consumption monitored by the sub meters. This extracted attribute provides a comprehensive view of device consumption.

D. Machine Learning Algorithms

Three regression algorithms were selected to represent complementary bias–variance and computational trade-offs. Models were implemented in scikit-learn (single-node baseline) and Apache Spark MLlib (distributed path) on the same 23-dimensional feature vector.

1) Linear Regression: Ordinary Least Squares Linear Regression (OLS) served as the baseline model, providing a simple, interpretable reference point. The model assumes a linear relationship between features and target variable:

$$\hat{y} = \beta_0 + \sum_{i=1}^{n} \beta_i x_i + \epsilon \tag{2}$$

2) Random Forest Regression: A random forest is an ensemble learning method that constructs multiple decision trees on random subsets of features and samples, and aggregates predictions by averaging them. This algorithm reduces variance through bootstrap aggregation and random feature subsampling.

$$\hat{y} = \frac{1}{T} \sum_{t=1}^{T} f_t(x) \tag{3}$$

3) Gradient Boosting Regression: Gradient Boosting constructs an ensemble of weak learners sequentially, with each new learner focusing on correcting the errors of previous learners. The model is built additively:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x) \tag{4}$$

E. Experimental Setup

1) Train-Test Split: The dataset was partitioned into training and testing sets using a time-series split to preserve temporal ordering. The first 80% of chronologically ordered data (1,639,404 samples) was allocated for training, while the remaining 20% (409,852 samples) was reserved for testing. This approach ensures that the model is evaluated on future time periods not seen during training, simulating real-world deployment scenarios.

2) **Feature Scaling:** StandardScaler was applied to normalize feature distributions, transforming each feature to have zero mean and unit variance:

$$x' = \frac{x - \mu}{\sigma} \tag{5}$$

3) **Evaluation Metrics:** Four evaluation metrics were employed to assess model performance comprehensively: Root Mean Square Error (RMSE):

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{6}$$

Mean Absolute Error (MAE):

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{7}$$

Coefficient of Determination (R2):

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{8}$$

Mean Absolute Percentage Error (MAPE):

$$MAPE = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \quad (9)$$

4) Implementation Environment: The project was implemented in Google Colab using Python with the scikit-learn machine learning package, pandas for data processing, and matplotlib/seaborn for graphical representation. The PySpark MLlib implementation is provided for distributed model training. All experiments were performed on Colab environment.

IV. Experimental Results and Discussion

A. Dataset Characteristics Analysis

Analysis of the University of California, Irvine's household energy consumption dataset reveals distinct time patterns that can be used for feature engineering and model design.

Figure 1 illustrates consumption patterns on daily, weekly, and monthly timescales.

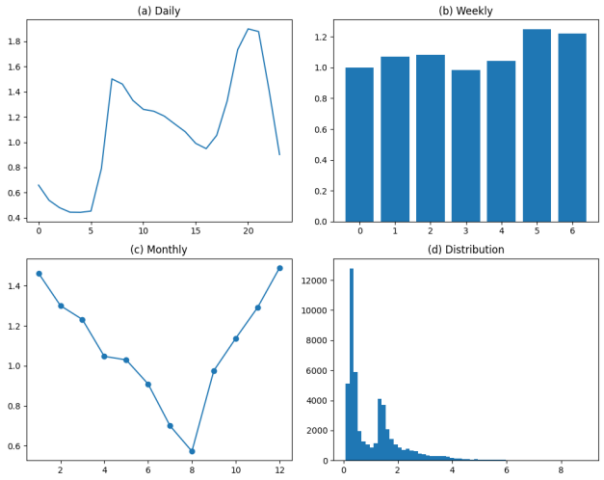

Fig. 1. Dataset characteristics: (a) Daily consumption pattern showing peaks during morning and evening hours, (b) Weekly pattern indicating higher consumption on weekends, (c) Monthly seasonal variations, (d) Distribution of global active power consumption values.

The daily consumption pattern (Fig. 1a) exhibits characteristic bimodal distribution with morning peaks (7-9AM) corresponding to household activity and evening peaks (6-9 PM) associated with cooking, heating, and entertainment usage. The weekly pattern (Fig. 1b) shows elevated consumption on weekends, reflecting increased occupancy and activity levels. Monthly variations (Fig.1c) indicate seasonal effects with higher consumption during winter months due to heating requirements. The consumption distribution (Fig. 1d) demonstrates right-skewed characteristics typical of residential energy usage.

B. Model Performance Comparison

Table II summarizes test-set performance. Gradient Boosting achieved the lowest RMSE (0.0295 kW) and highest R^2 (0.9989), a 21.1% RMSE reduction relative to Linear Regression (0.0374 kW). Random Forest attained the lowest MAE (0.0173 kW) and MAPE (2.31%). All models exceeded R^2 = 0.998, indicating that minute-level lag and rolling features capture strong autocorrelation in household load.

TABLE II
Model Performance Comparison

Model	RMSE	MAE	R^2	MAPE (%)	Train (s)
Linear Regression	0.0374	0.0232	0.9983	3.67	4.37
Random Forest	0.0326	0.0173	0.9987	2.31	356.71
Gradient Boosting	0.0295	0.0174	0.9989	2.60	626.77

TABLE III
Spark MLlib Training Time

Spark Model	RMSE	Training (s)
Spark Linear Regression	0.0373	44.55
Spark Random Forest	0.0805	217.33
Spark GBT	0.0795	593.64

Speedup S=Tsklearn /Tspark : LR 0.10, RF 1.64, GBT 1.06 Spark LR matched sklearn RMSE; Spark tree models trained faster but with higher RMSE under default MLlib settings on local[*]—discuss in Limitations.

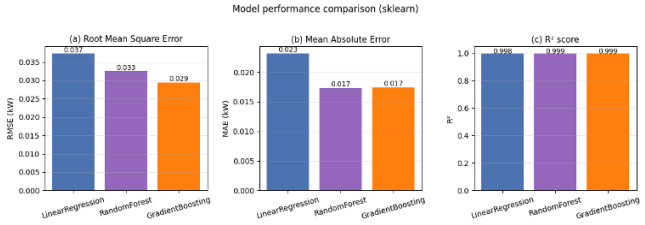

Fig. 2. Model performance comparison across evaluation metrics: (a) Root Mean Square Error, (b) Mean Absolute Error, (c) Coefficient of Determination (R^2).

C. Computational Performance Analysis

Linear Regression completed training in 4.37 s, approximately 82× faster than Random Forest (356.71 s) and 143× faster than Gradient Boosting (626.77 s). Dataset ingestion required 160.4 s for 2,049,280 raw minute-level records (~12,775 rows/s), indicating that end-to-end processing workflows at utility scale benefit from distributed ingest (e.g., Spark) even when single-node learners are fast.

Tree ensembles improve predictive accuracy at substantially higher training cost. For applications requiring frequent model retraining, near-real-time scoring after offline training, or deployment on resource-constrained edge nodes, Linear Regression offers a pragmatic latency–accuracy trade-off despite modestly higher RMSE (0.0374 kW vs. 0.0295 kW for Gradient Boosting).

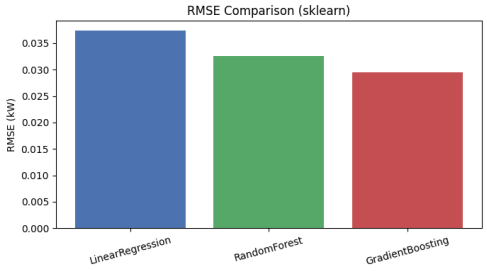

Fig. 3. Training and prediction time comparison across models.

D. Feature Importance Analysis

Permutation or impurity-based importance from Random Forest and Gradient Boosting indicated that Global_intensity, Voltage, and lagged global active power contributed most to predictions, consistent with the physical relationship $P = V \times I \times \cos(\phi)$.

Cyclical time features and rolling means captured diurnal structure (morning/evening peaks). Sub-metering channels contributed modest incremental gain, suggesting that circuit-level loads refine but do not dominate minute-ahead forecasts when global electrical measurements and lags are available.

E. Scalability Analysis

1) Data Size Scaling: Scalability analysis examined how algorithm training time scales with increasing data volumes. Using Linear Regression as a representative algorithm, training was performed on data fractions of 25%, 50%, 75%, and 100%.

TABLE IV
Data Size Scaling Results (Linear Regression)

Data Fraction	Samples	Time (s)	Scaling Factor
25%	409,851	0.36	1.00
50%	819,702	1.09	1.51
75%	1,229,553	2.13	1.96
100%	1,639,404	2.58	1.78

The scaling analysis reveals near-linear scaling characteristics with a scaling factor approaching 1.78 for the full dataset. This indicates that Linear Regression training time increases proportionally with data size, demonstrating excellent scalability.

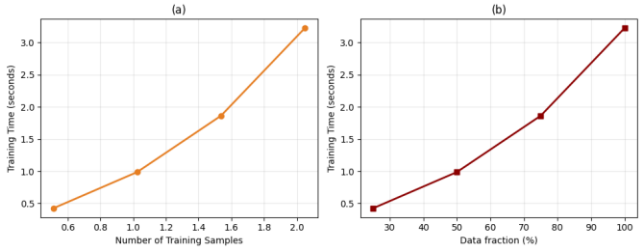

Fig. 4. Data size scalability analysis: (a) Training time versus number of training samples, (b) Training time versus data fraction percentage.

The scalability diagram (Fig. 4) illustrates near-linear scaling behavior for Linear Regression. The slight deviation from ideal linear growth (scaling factor 1.78 at 100% of the training set) is attributed to memory hierarchy effects and Colab runtime overhead rather than algorithmic quadratic complexity.

2) Model Complexity Scaling: The complexity scaling of the random forest was evaluated by comparing configurations with different numbers of estimators and varying maximum depths:

- Configuration 1: 10 trees, maximum depth 5 – Training time: 34.01 seconds, test RMSE: 0.0577 kW
- Configuration 2: 30 trees, maximum depth 10 – Training time: 148.01 seconds, test RMSE: 0.0326 kW

A 4.35-fold increase in training time (from 34.01 s to 148.01 s) occurred when the number of trees increased threefold (10 → 30) and the maximum depth doubled (5 → 10).

This reflects super-linear scaling with model complexity for Random Forest on the full training set (1,639,404 samples). The corresponding 43.5% reduction in test RMSE (from 0.0577 kW to 0.0326 kW) shows that additional computational cost yields substantial accuracy gains.

F. Discussion

1) **Comparison between accuracy and computation:** Experimental results show a clear trade-off between prediction quality and training cost. Gradient Boosting achieved the best test performance (RMSE 0.0295 kW, R^2

0.9989) but required the longest training time (626.77 s). Random Forest offered an intermediate compromise (RMSE 0.0326 kW, 356.71 s). Linear Regression trained in 4.37 s with modestly higher error (RMSE 0.0374 kW). For operational smart grid analytics, algorithm choice should reflect deployment constraints:

Low-latency inference or frequent retraining: Linear Regression, or Random Forest with reduced n_estimators and max_depth;

Offline batch forecasting with highest accuracy: Gradient Boosting;

Balanced accuracy and cost: Random Forest with tuned hyperparameters (e.g., 30 trees, depth 10).

Apache Spark MLlib reproduced linear-model accuracy (RMSE 0.0373 kW) but tree models showed higher RMSE under default settings on a single-node Colab session; cluster-scale tuning is needed for production AMI workloads.

2) Impact of feature engineering: Strong performance across all models ($R^2 > 0.998$) indicates that cyclical time encoding, minute-level lag features, and rolling statistics capture dominant short-term autocorrelation in minute-resolution load data. Sub-metering channels provided complementary information on circuit-level consumption, while global electrical measurements and lags carried most of the predictive signal.

3) Scalability results: Linear Regression exhibited near-linear growth in training time with data volume (0.36 s at 25% to 2.58 s at 100% of the training set; Fig. 4). Dataset ingestion of approximately 2.05 million records required 160.4 s on Colab. Together with the Spark implementation path, these results support feasibility for larger AMI archives when ingestion and training are distributed, although wall-clock gains depend on cluster size and partitioning.

4) Comparison with previous work: The best sklearn RMSE (0.0295 kW) is competitive with published results on the UCI household dataset when differences in resolution (minute vs. hour), forecast horizon, feature sets, and train/test protocols are considered. Direct numeric comparison with prior studies requires matching experimental conditions; this work emphasizes reproducible baselines (sklearn and Spark) on a public dataset with explicit chronological evaluation.

5) Limitations: Several limitations should be noted. 1) A single household in Sceaux, France limits generalization to other dwellings and climates. 2) Weather and tariff variables were not included, which may affect heating-dominated periods. 3) Experiments used Google Colab with scikit-learn and Spark in local[*] mode, not a multi-node cluster. 4) Spark Random Forest and GBT underperformed sklearn trees in RMSE while reducing training time—hyperparameter optimization and cluster deployment are required before operational conclusions. 5) High R^2 may partly reflect autocorrelation from lag features; future work should report horizon-specific metrics and leakage-safe validation.

V. Conclusion

A. Summary of Findings

This study presented a comparative analysis of machine learning algorithms for residential global active power forecasting in smart grid analytics, using Big Data–scale AMI data and reproducible processing workflow. The principal findings are:

1) Algorithm performance: Gradient Boosting achieved the best test-set accuracy (RMSE 0.0295 kW, R^2 0.9989), outperforming Random Forest (RMSE 0.0326 kW, R^2 0.9987) and Linear Regression (RMSE 0.0374 kW, R^2 0.9983). Random Forest achieved the lowest MAPE (2.31%) and MAE (0.0173 kW).

2) Computational efficiency: Linear Regression trained in 4.37 s, approximately 82× faster than Random Forest (356.71 s) and 143× faster than Gradient Boosting (626.77 s). Dataset ingestion of 2,049,280 records required 160.4 s, supporting the need for distributed ingest at utility scale.

3) Distributed implementation: The same feature engineering workflow was implemented in Apache Spark MLlib on Google Colab (local[*]). Spark Linear Regression matched sklearn accuracy (RMSE 0.0373 kW), validating the distributed workflow; Spark tree models trained faster but showed higher RMSE under default hyperparameters, indicating that cluster-scale tuning is required for production parity.

4) Feature engineering: Cyclical calendar encoding, minute-level lag features, and rolling statistics enabled all models to exceed $R^2 > 0.998$, confirming that short-term persistence and diurnal structure dominate minute-level residential load.

5) Scalability: Linear Regression training time grew predictably with training-set size (Fig. 4); Random Forest cost increased sharply with n_estimators and max_depth, with 43.5% RMSE improvement between the two configurations tested. These results support distributed training for large meter populations.

B. Practical Recommendations

Based on the experiments, we recommend:

Real-time or resource-constrained scoring: Linear Regression (RMSE 0.0374 kW; fastest training).

Accuracy-critical batch forecasting (e.g., demand response planning): Gradient Boosting (lowest RMSE; accept higher training cost).

Balanced accuracy–cost trade-off: Random Forest (competitive RMSE and MAPE).

City-scale data volumes: Spark MLlib batch training workflows with partitioned ingest and scheduled retraining.

Modeling practice: Prioritize cyclical time encoding and lag/rolling features before increasing model complexity.

C. *Future Work*

Future research should address: (i) multi-household and multi-site training; (ii) weather and tariff covariates; (iii) controlled comparison with deep sequence models (LSTM/GRU) on identical splits; (iv) multi-node Spark clusters with measured speedup and cost; and (v) online or incremental learning for drifting consumption patterns.

D. *Concluding Remarks*

The transition toward smart grids and AMI generates unprecedented consumption data volumes, enabling data-driven forecasting and operational optimization. This work demonstrates that interpretable machine learning models with careful temporal feature design can achieve excellent minute-level accuracy while exposing clear accuracy–latency trade-offs relevant to real-world deployment.

The source code and dataset used in this study are available at:

https://github.com/doaaabdeen/BigData

REFERENCES

[1] T. Hong and J. Fan, "Probabilistic electric load forecasting: A tutorial review," Int. J. Forecasting, vol. 32, no. 3, pp. 914–938, 2016.

[2] E. Hossain, I. Khan, F. Un-Noor, S. S. Sikander, and M. S. H. Sunny, "Application of big data and machine learning in smart grid, and associated security concerns: A review," IEEE Access, vol. 7, pp. 13960–13988, 2019.

[3] E. C. U. Eneh, T. O. Akinbulire, and P. A. A. Atayero, "Big data analytics in smart grids: A review," Eng. Rep., vol. 3, no. 6, e12278, 2021.

[4] H. V. Jagadish et al., "Big data and its technical challenges," Commun. ACM, vol. 57, no. 7, pp. 86–94, 2014.

[5] M. Zaharia et al., "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in Proc. USENIX NSDI, 2012, pp. 15–28.

[6] X. Meng et al., "MLlib: Machine learning in Apache Spark," J. Mach. Learn. Res., vol. 17, no. 34, pp. 1–7, 2016.

[7] M. Assefi et al., "Big data machine learning using Apache Spark MLlib," in Proc. IEEE Int. Conf. Big Data, 2017, pp. 3492–3498.

[8] A. Abdel Hai and B. Forouraghi, "On scalability of distributed machine learning with big data on Apache Spark," in Proc. IEEE Int. Congr. Big Data, 2018, pp. 117–124.

[9] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[10] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Ann. Statist., vol. 29, no. 5, pp. 1189–1232, 2001.

[11] S.-V. Oprea and A. Bâra, "Machine learning algorithms for short-term load forecast in residential buildings using smart meters, sensors and big data solutions," IEEE Access, vol. 7, pp. 157104–157116, 2019.

[12] J.-S. Chou and D.-S. Tran, "Forecasting energy consumption time series using machine learning techniques based on usage patterns of residential households," Energy, vol. 165, pp. 709–726, 2018.

[13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Comput., vol. 9, no. 8, pp. 1735–1780, 1997.

[14] P. V. B. Ramos et al., "Residential energy consumption forecasting using deep learning models," Appl. Energy, vol. 348, p. 121456, 2023.

[15] S.-J. Bu and S.-B. Cho, "Time series forecasting with multi-headed attention-based deep learning for residential energy consumption," Energies, vol. 13, no. 18, p. 4722, 2020.

[16] B. Dhupia, M. U. Rani, and A. Alameen, "The role of big data analytics in smart grid management," in Emerging Research in Data Engineering Systems and Computer Communications, Springer, 2020, pp. 475–490.

[17] G. Zhang and J. Guo, "A novel ensemble method for hourly residential electricity consumption forecasting by imaging time series," Energy, vol. 203, p. 117858, 2020.

[18] N. Mostafa, H. S. M. Ramadan, and O. Elfarouk, "Renewable energy management in smart grids by using big data analytics and machine learning," Mach. Learn. Appl., vol. 10, p. 100397, 2022.

[19] S. Kumari, N. Kumar, and P. S. Rana, "Big data analytics for energy consumption prediction in smart grid using genetic algorithm and LSTM," Comput. Inform., vol. 40, no. 5, pp. 1074–1103, 2021.

[20] D. Hebrail and Y. Berard, "Individual household electric power consumption," UCI Machine Learning Repository, 2012. [Online]. Available: https://archive.ics.uci.edu/dataset/235/individual+household+electric+power+consumption.
doi: 10.24432/C5K89G.